

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

Старший преподаватель

должность, уч. степень, звание

подпись, дата

С.Ю. Гуков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

Сортировки. Определение сложности алгоритма

по курсу: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1 Цель работы	3
2 Задание.....	4
3 Краткое описание хода работы	6
4 Исходный код.....	7
5 Результат работы с примерами	10
6 Выводы	12

1 Цель работы

Цель данной лабораторной работы заключается в углублённом изучении работы различных алгоритмов сортировки, таких как пузырьковая сортировка, сортировка расческой, быстрая сортировка и сортировка слиянием. В рамках работы будет проведён детальный анализ принципов их функционирования, что позволит понять, как каждый алгоритм обрабатывает данные и какие методы используются для упорядочивания элементов. Особое внимание будет уделено оценке временной и пространственной сложности этих алгоритмов, чтобы уметь сравнивать их эффективность в зависимости от объёма и структуры входных данных.

2 Задание

Для выполнения лабораторной работы необходимо разработать программу, реализующую сортировку слов из заданного текстового файла и их дальнейший анализ в соответствии с выбранными параметрами варианта. Программа должна принимать путь к исходному текстовому файлу через консоль и считывать текст с минимальными погрешностями, разделяя его на отдельные слова. Знаки препинания и символы, не входящие в алфавит, не учитываются при разбиении текста на слова. Длина слова определяется количеством букв в нем, числа могут учитываться или игнорироваться в зависимости от условий варианта.

После разбиения текста на массив слов, необходимо выполнить сортировку в соответствии с параметрами, заданными в варианте (по алфавиту или по длине слова). Отсортированный массив выводится в выходной файл, например, «result.txt». В консоль выводить результат не требуется.

Программа должна также выполнить простой анализ текста и вывести следующую информацию в консоль и файл, например, «analysis.txt»:

- Исходный текст,
- Параметры выбранного варианта,
- Количество слов в исходном тексте,
- Время выполнения сортировки,
- Количество слов на каждую букву алфавита,
- Количество слов каждой длины,

Необходимо провести 10 тестов с разными по длине текстами, чтобы оценить влияние объема текста на время сортировки. Длина текстов должна варьироваться от 1 000 до 130 000 символов. Для каждого теста должны быть созданы три файла: входной текст, отсортированный массив и файл анализа.

Требуется представить график или диаграмму зависимости времени выполнения сортировки от количества слов в тексте, а также оценить и обосновать сложность алгоритма в O-нотации.

Вариант 2.

- Использование латиницы,
- Критерий сортировки: по количеству символом в слове,
- Сортировка расческой,
- Порядок сортировки: по убывания,
- Игнорировать числа,

3 Краткое описание хода работы

В данной работе была реализована сортировка слов в предложении на латинице с использованием метода сортировки "расческа". Основной целью было отсортировать слова по количеству символов в них, игнорируя все числовые значения. В процессе работы использовались регулярные выражения для фильтрации слов и их символов, а также реализована функция для сортировки, которая учитывает порядок по убыванию длины слов. Таким образом, все слова были обработаны и отсортированы в соответствии с заданными критериями.

Основная логика программы заключалась в последовательной обработке текста: сначала выполнялась сортировка слов с удалением лишних символов, затем осуществлялся подсчет количества повторяющихся букв в отсортированном массиве, переводя каждую букву в Unicode для сортировки расческой. Итоговые результаты записывались в текстовые файлы, что позволило проанализировать входные данные и эффективность работы алгоритма. Временные затраты на выполнение программы фиксировались, что дало возможность оценить производительность используемого метода сортировки

4 Исходный код

Ниже предоставлен исходный код моей программы с комментариями основных блоков:

```
//Сортировка Расческа
function combSorting(array) {
    var interval = Math.floor(array.length / 1.25);
    while (interval > 0) {
        for (var i = 0; i + interval < array.length; i++) {
            if (array[i] > array[i + interval]) {
                var small = array[i + interval];
                array[i + interval] = array[i];
                array[i] = small;
            }
        }
        interval = Math.floor(interval / 1.25);
    }
    return array;
}

//Сортировка текста по длине слова, где числа не учитываются, по убыванию
function sortText(text) {
    var words = text.split(' ');
    var sortedWords = [];
    for (var i = 0; i < words.length; i++) {
        var word = words[i];
        var latinWord = word.replace(/^[^a-z]/gi, "");
        if (latinWord.length > 0) {
            sortedWords.push(latinWord);
        }
    }
}
```

```

sortedWords.sort(function (a, b) {
    return b.length - a.length;
});
return sortedWords.join(' ').toLowerCase();
};

```

// Функция для сортировки предложения на латинице побуквенно без лишних знаков.

```

function stringToArrayUnicode(string) {
    var array = [];
    for (var i = 0; i < string.length; i++) {
        if ((string[i]).match(/[a-z]/)) {
            array.push((string[i]).charCodeAt(0));
        }
    }
    return array;
}

```

//Функция для вывода количества повторяющихся букв в предложении на основе сортировки расческой.

```

function outputRepeatLetters(text) {
    let arrayUnicode = combSorting(stringToArrayUnicode(sortText(text)));
    let setUnicode = [...new Set(arrayUnicode)];
    let output = "";
    const latinAlphabet = 'abcdefghijklmnopqrstuvwxyz';
    for (let i = 97; i < latinAlphabet.length + 97; i++) {
        let repeat = 0;
        for (let j = 0; j < setUnicode.length; j++) {
            if (i == setUnicode[j]) {
                repeat = arrayUnicode.filter(num => num === i).length;

```



```

    }
  }
  output += (String.fromCharCode(i) + ' - ' + repeat + '\n');
}
return output;
}

//Запуск счетчика времени выполнения программы.
const start = new Date().getTime();
const fs = require('fs');
//Чтение и обновление файла
fs.writeFileSync('analysis.txt', '');
let randomLatinText = fs.readFileSync('original.txt', 'utf8');
sortedSentence = sortText(randomLatinText)
fs.writeFileSync('result.txt', sortedSentence);
fs.appendFileSync('analysis.txt', 'Введенный текст:\n' + randomLatinText + '\n' +
'Вариант 2: Латиница, По кол-ву символов в слове, по убыванию,
игнорировать числа, Сортировка Расчетка\nКоличество слов: ' +
((sortedSentence.split(' ').length + '\n'));
//Запуск сортировки
let output = outputRepeatLetters(randomLatinText);
//Окончание счетчика времени выполнения программы.
const end = new Date().getTime();
fs.appendFileSync('analysis.txt', ('Время выполнения программы: ' + (end - start)
+ 'ms\n'));
fs.appendFileSync('analysis.txt', output);

```

5 Результат работы с примерами

В файле `original.txt` приведен текст, который программа будет анализировать.

1 Lorem ipsuam dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas

Рисунок 1 – original.txt

В файле result.txt приведен текст, который отсортирован строго по пунктам выбранного задания.

1	conspectetur	pellentesque	consectetur	pellentesque	consectetur	pellentesque	solicitudin	pellentesque	pellentesque	pellentesque	pellentesque	pellentesque	pellentesque
	pellentesque	pellentesque	solicitudin	consectetur	pellentesque	solicitudin	pellentesque	pellentesque	pellentesque	pellentesque	consectetur	consectetur	pellentesque
	pellentesque	pellentesque	consectetur	pellentesque	solicitudin	pellentesque	pellentesque	pellentesque	pellentesque	pellentesque	pellentesque	pellentesque	solicitudin
	consectetur	pellentesque	solicitudin	pellentesque	pellentesque	pellentesque	consectetur	consectetur	pellentesque	consectetur	consectetur	pellentesque	consectetur
	pellentesque	solicitudin	pellentesque	pellentesque	pellentesque	pellentesque	pellentesque	pellentesque	pellentesque	solicitudin	consectetur	pellentesque	solicitudin
	pellentesque	pellentesque	pellentesque	pellentesque	consectetur	consectetur	pellentesque	consectetur	consectetur	pellentesque	solicitudin	pellentesque	solicitudin
	pellentesque	pellentesque	pellentesque	pellentesque	pellentesque	pellentesque	solicitudin	consectetur	pellentesque	solicitudin	pellentesque	pellentesque	pellentesque

Рисунок 2 – result.txt

В файле analysis.txt приведен результат задания, с полным анализом текста.

```

Введенный текст:
uis, nisi. Curabitur ligula sapien, tincidunt non, euismod vitae, posuere imperdiet, leo. Maecenas malesuada. Praesent congue erat at massa. Sed cursus turpis vitae tortor. Donec
posuere vulputate arcu. Phasellus accumsan cursus velit. Vestibulum ante ipsum primis in
Вариант 2: Латиница, По кол-ву символов в слове, по убыванию, игнорировать числа, Сортировка Расчета
Количество слов: 9998
Время выполнения программы: 52ms
a - 4390
b - 736
c - 2363
d - 1557
e - 6537
f - 449
g - 793
h - 335
i - 5512
j - 60
k - 1
l - 3438
m - 2554
n - 3475
o - 2320
p - 1640
q - 711
r - 3030
s - 4894
t - 4412
u - 5263
v - 913
w - 0
x - 0
y - 38
z - 0

```

Рисунок 3 – analysis.txt

На четвертом рисунке изображен график сложности алгоритма на основе 10 тестов, содержащие от 1 000 символом до 130 000 символов.

Сложность времени вывода повторяющихся букв в зависимости от количества символов

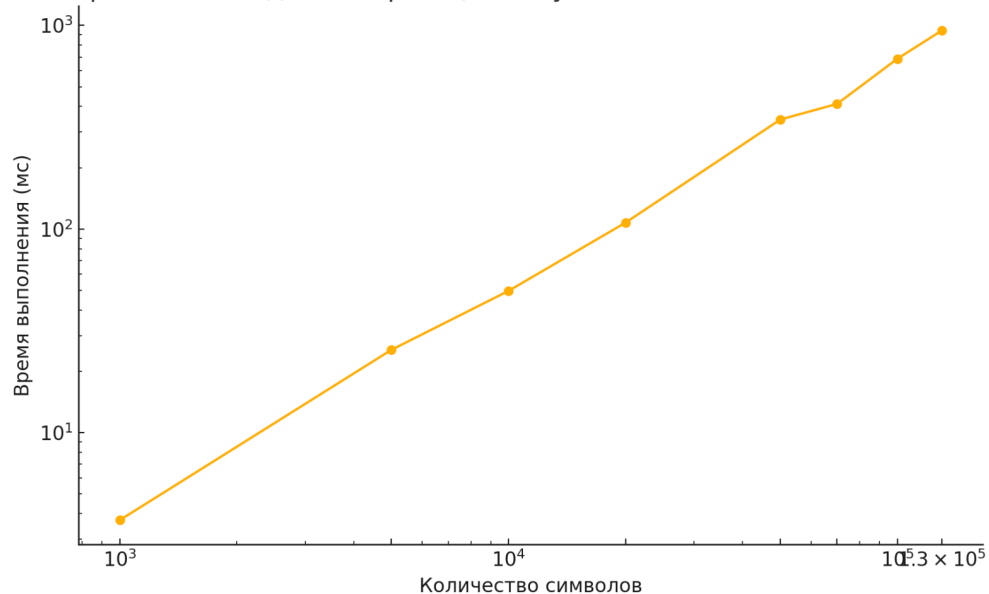


Рисунок 4 – График сложности алгоритма на основе 10 тестов

6 Выводы

В данной лабораторной работе была проведена реализация и анализ алгоритма сортировки на основе метода "расческа". Основной целью было отсортировать слова в текстовом файле по их длине, игнорируя числовые значения, и проанализировать время выполнения алгоритма в зависимости от объема входных данных.

Реализация алгоритма: Программа успешно выполнила сортировку текста на латинице с использованием сортировки "расческа". В процессе работы использовались регулярные выражения для фильтрации слов, что позволило достичь высокой точности в разбиении текста.

Временные затраты: Временные затраты на выполнение алгоритма были зафиксированы для десяти тестов с различными размерами текста, варьирующимися от 1000 до 130000 символов. Результаты замеров времени были визуализированы на графике.

Временная сложность данного алгоритма сортировки в худшем и среднем случаях составляет $O(n^2)$, где n — это количество элементов в массиве. Однако в лучшем случае (если массив уже отсортирован) сложность может быть $O(n)$. Таким образом, в общем случае сложность можно оценить как $O(n \log n)$, учитывая улучшение по сравнению с традиционными сортировками.

Подсчет повторяющихся букв основан на преобразовании текста в массив кодов символов и сортировке. Это также имеет временную сложность около $O(n \log n)$ из-за необходимости сортировки массива.

Логарифмическая шкала на осях X и Y позволяет лучше визуализировать эту зависимость, показывая, что при увеличении объема данных время выполнения возрастает, хотя и не пропорционально.

Таким образом, полученные данные и графическое представление подтверждают, что алгоритм сортировки "расческа" подходит для обработки текстов средней длины, однако для очень больших объемов данных могут потребоваться более эффективные алгоритмы сортировки, такие как быстрая

сортировка или сортировка слиянием, чтобы минимизировать время выполнения.